

LEVEL 14 — Apply, Map and Transform

Python + Pandas | Examples with output, line by line

Important Note

map() is commonly used with Python iterables and Pandas Series. **apply()** applies a function to a Series or along a DataFrame axis. **applymap()** was the older Pandas name for applying a function element-by-element to a DataFrame; in modern Pandas, use **DataFrame.map()** instead. **transform()** keeps the original shape and is especially useful for group-wise transformations.

1. map()

```
numbers = [1, 2, 3, 4]
result = list(map(lambda x: x * 2, numbers))
print(result)
```

```
# Output:
[2, 4, 6, 8]
```

map() applies a function to every item of an iterable. It returns a map object, so list() is often used to display the result.

2. apply() with a Series

```
import pandas as pd

s = pd.Series([10, 20, 30])
result = s.apply(lambda x: x + 5)
print(result)
```

```
# Output:
0    15
1    25
2    35
dtype: int64
```

Series.apply() calls the function for each value in the Series.

3. DataFrame element-wise operations

```
df = pd.DataFrame({
    "A": [1, 2],
    "B": [3, 4]
})

result = df * 10
print(result)
```

```
# Output:
   A  B
0  10 30
1  20 40
```

For simple arithmetic, Pandas/NumPy vectorized operations are preferred because they are concise and generally faster than Python-level function calls.

4. DataFrame.map() / older applymap()

```
df = pd.DataFrame({
    "A": [1, 2],
    "B": [3, 4]
})

result = df.map(lambda x: x * 2)
print(result)
```

```
# Output:  
  A  B  
0  2  6  
1  4  8
```

DataFrame.map() applies a function to every individual cell. In older Pandas versions this operation was called DataFrame.applymap().

5. transform()

```
s = pd.Series([10, 20, 30])  
result = s.transform(lambda x: x + 100)  
print(result)
```

```
# Output:  
0    110  
1    120  
2    130  
dtype: int64
```

transform() returns a result aligned to the original shape. This is particularly valuable with grouped data.

6. Lambda functions

```
square = lambda x: x ** 2  
  
print(square(5))
```

```
# Output:  
25
```

A lambda is a small anonymous function. Syntax: lambda arguments: expression.

7. Function application to a Series

```
def add_tax(x):  
    return x * 1.18  
  
prices = pd.Series([100, 200, 300])  
result = prices.apply(add_tax)  
print(result)  
  
# Output:  
0    118.0  
1    236.0  
2    354.0  
dtype: float64
```

A named custom function can be passed directly to Series.apply().

8. Function application to a DataFrame

```
df = pd.DataFrame({  
    "A": [10, 20],  
    "B": [30, 40]  
})  
  
def total(column):  
    return column.sum()  
  
result = df.apply(total)  
print(result)  
  
# Output:  
A    30  
B    70  
dtype: int64
```

By default, DataFrame.apply() works column-by-column (axis=0).

9. DataFrame apply() row-wise

```
df = pd.DataFrame({
    "A": [10, 20],
    "B": [30, 40]
})

result = df.apply(lambda row: row["A"] + row["B"], axis=1)
print(result)

# Output:
0    40
1    60
dtype: int64
```

axis=1 passes each row to the function. axis=0 (default) passes each column.

10. Custom functions

```
def grade(score):
    if score >= 90:
        return "A"
    elif score >= 75:
        return "B"
    else:
        return "C"

marks = pd.Series([95, 82, 65])
result = marks.apply(grade)
print(result)

# Output:
0    A
1    B
2    C
dtype: object
```

Custom functions are useful when the transformation needs multiple steps or conditions.

11. GroupBy + transform() — Very Important

```
df = pd.DataFrame({
    "Department": ["IT", "IT", "HR", "HR"],
    "Salary": [50000, 70000, 40000, 60000]
})

df["Dept_Avg"] = df.groupby("Department")["Salary"].transform("mean")
print(df)
```

Output:

	Department	Salary	Dept_Avg
0	IT	50000	60000.0
1	IT	70000	60000.0
2	HR	40000	50000.0
3	HR	60000	50000.0

Unlike a normal groupby aggregation that reduces each group to one row, `transform()` broadcasts the group result back to every original row.

12. Quick Difference Table

Tool	Works on	Main idea	Shape
<code>map()</code>	Python iterable / Series	Apply function to each item/value	Usually same item count
<code>Series.apply()</code>	Series	Apply function to values	Usually same length
<code>DataFrame.apply()</code>	DataFrame	Apply along rows or columns	Can change shape
<code>DataFrame.map()</code>	DataFrame	Apply to each cell	Same shape
<code>transform()</code>	Series / DataFrame / GroupBy	Transform while preserving alignment	Same shape
<code>lambda</code>	Any compatible callable	Small anonymous function	Depends on use

13. When to use what?

Use `map()` when you want to transform each item/value individually.

Use `Series.apply()` for a custom function on a Series.

Use `DataFrame.apply()` when the function needs a whole row or column.

Use `DataFrame.map()` when every cell needs the function independently.

Use `transform()` when you need transformed values that stay aligned with the original data, especially after groupby.

Use vectorized operations such as `df * 2`, `df['A'] + 10`, `np.sqrt(df['A'])` whenever possible.

14. Practice Questions

1. Double every value in a Series using `map()`.
2. Convert a Series of names to uppercase using `apply()`.
3. Add 10 to every DataFrame cell using `DataFrame.map()`.
4. Find row-wise totals using `DataFrame.apply(..., axis=1)`.
5. Create a custom function that labels marks as Pass/Fail.
6. Use `groupby().transform("mean")` to calculate each employee's department average salary.
7. Explain why `transform()` is different from `groupby().mean()`.

Key takeaway

The most important distinction is **shape and level of application**: `map` works item-by-item, `apply` can work on Series values or complete DataFrame rows/columns, `DataFrame.map` works cell-by-cell, and `transform` is designed to return values aligned with the original data.